CSS

Media Queries, Frameworks, Preprocessors, Advanced Concepts

paged media, component-based and utility-based frameworks, CSS preprocessors, Animations

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Media Queries

Media Type, Media Features, Printed Media

Media Queries

- allows styles to apply only under certain conditions
- use the **@media** rule for conditional style definitions
- Media Queries Level 3 (W3C Recommendation)
  - Media Queries Level 4 (W3C Candidate Recommendation Draft)

```
@media not|only mediatype and (expressions) {
  /* CSS rules */
}
```

- **not**: negates the entire rule
- **only**: older browsers will ignore the construction
- **mediatype**: type of media/device
  - screen, print, speech, all, …
- **expressions**: conditions for the rule (screen size, …) – **media features**

Media Types (historical)

- **defined in CSS Level 2**
  - tty – for teletype terminals, character-based displays
  - tv – for television devices, low-resolution screens
  - projection – for projectors / presentations
  - handheld – for early mobile devices / PDAs
  - braille – for refreshable braille displays
  - embossed – for braille printers
  - aural – for speech synthesizers / screen readers
  - speech – modern replacement for aural, largely unsupported
- **obsolete HW** – devices like teletype, braille embossers, or PDAs are largely gone.
- **device convergence** – modern devices combine features of old categories
- **limited browser support** – legacy types were rarely or never fully supported
- **better alternatives available** – media features (width, pointer, hover, prefers-*)

Media Types (current)

- **Media Queries Level 4 uses the following types:**
  - all – applies to all devices and media (default)
  - screen – for computer screens, tablets, smartphones, etc.
  - print – for printed documents and print preview

```
/* import CSS file based on media type at the beginning of another CSS file */
@import "screen.css" screen;
@import "print.css" print;
@import "common.css" all;
```

```
<!-- link CSS file based on media type -->
<link rel="stylesheet" type="text/css" media="print">
```

- they use **media features** for responsiveness

Media Features: Dimension

- width – viewport width
- height – viewport height
- min-width / max-width – min/max viewport width (most common for breakpoints)
- min-height / max-height – min/max viewport height

- device-width / device-height-- dimensions of the entire device screen (less commonly used)

- the modern solution that makes using device-width unnecessary in most cases:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Media Features: Other

- orientation – `portrait` or `landscape`
- resolution – pixel density (dpi or dppx)
- aspect-ratio – width-to-height ratio of the viewport
- pointer – type of pointing device (none, `coarse` – touchscreen, `fine` – mouse. stylus)
- hover – whether the primary input can hover (none / `hover`)
- prefers-color-scheme – `light` or `dark` mode
- prefers-reduced-motion – prefers less animation
- prefers-contrast – high / low contrast

Printed Media

- targets printed documents and print preview in browsers.
- CSS rules are wrapped inside a media query to style content specifically for printing

```
@media screen {
    body {
        font-family: sans-serif;
        font-size: 12pt;
    }
}

@media print {
    body {
        font-family: serif;
        font-size: 10pt;
        color: black;
        background: white;
    }
    nav, .sidebar, .ads {
      display: none; /* hide unnecessary elements in print */
    }
}
```

Printed Media: Pagination

- **older CSS2 properties:**
  - page-break-before: `auto | always | avoid | left | right`
  - page-break-after: `auto | always | avoid | left | right`
  - page-break-inside: `auto | avoid`

```
…
<div class="slide">
    Slide content
</div>
…
```

```
div.slide {
    page-break-after: always; /* break after */
    page-break-inside: avoid; /* avoid breaking inside */
}
```

Printed Media: Pagination

- **CSS Fragmentation Module Level 3 (W3C Candidate Recommendation)**
  - break-before: `auto` | `page` | `column` | `avoid` | `left` | `right`;
  - break-after: `auto` | `page` | `column` | `avoid` | `left` | `right`;
  - break-inside: `auto` | `avoid` | `avoid-page` | `avoid-column`;
- **values:**
  - auto – uses default behavior (breaking rules according to context)
  - page – inserts a page break
  - column – inserts a column break (in multi-column layout)
  - avoid – prevents element splitting if possible
  - left / right – for printing: inserts a page that will start on the left/right

- adds support for multi-column layouts (`column-count: 3;`)

Support for Muluti-column Layouts

- additionally, it supports multi-column layouts (`column-count: 3;`)

```
.container {
  column-count: 3;
  column-gap: 20px;
}

h2 {
  break-before: column;  /* every heading starts new column */
}

section {
  break-inside: avoid;   /* section will not be splitted into columns */
}
```

@page Rule

- rule which ses properties of printed pages
- **CSS Paged Media Module Level 3 (W3C Working Draft)**

```
@media print {
  @page {
    size: A4 landscape;  /* A4 page, landscape orientation */
    margin: 2cm;         /* page margins */
  }

  @page :first {
    margin-top: 4cm;     /* extra space for title page */
  }

  @page :left {
    margin-left: 3cm;    /* wider left margin */
  }

  @page :right {
    margin-right: 3cm;   /* wider right margin */
  }
}
```

Printed Media: Recommendation

- **Hide non-essential elements:** display: none for navigation, ads, and sidebars.
- **Control page breaks:** use break-before, break-after, or break-inside: avoid for sections.
- **Optimize typography:** adjust font-size, line-height, and font-family for readability.
- **Simplify colors:** set color: black and background: white for legibility and ink saving.
- **Manage images:** scale or hide large images to fit pages neatly.
- **Set page size and margins:** use @page rule (size, margin) for consistent layout.
- **Avoid complex layouts:** remove floats, columns, or interactive elements that do not print well.
- **Test print preview:** always check how content breaks across pages and adjust accordingly.

CSS Frameworks

Component and Utility Based

Motivation

- **a pre-prepared set of CSS (and sometimes JavaScript) files that provides:**
  - predefined styles for layout, typography, and UI components
  - reusable design patterns to speed up development
  - responsive and cross-browser compatibility out of the box

- **goal:**
  - speed up development
  - ensure consistent design across pages and projects
  - promote responsive and cross-browser layouts
  - reduce repetitive CSS from scratch

Type 1: Component-based Frameworks

- provides ready-made UI components (buttons, forms, navigation, cards…)
- **Bootstrap**, **Foundation**, **Bulma**, **Materialize**
- 🟢 **advantages:**
  - quickly use pre-built elements
  - consistent look across the project
  - great for rapid prototyping
- 🔴 **disadvantages:**
  - less flexibility – customizing components for unique designs can be harder

Bootstrap

- getbootstrap.com, MIT License
- popular front-end framework with CSS, JS components, and responsive grid
- 🟢 **advantages:**
  - fast development, responsive layouts, consistent design, large community
- 🔴 **disadvantages:**
  - can be bloated, sites may look similar, custom overrides can be tricky

- **components**, **layout**

Example

Foundation

- get.foundation, MIT License
- responsive front-end framework with grid system, UI components, and accessibility features
- 🟢 **advantages:**
  - mobile-first, fully responsive layouts
  - strong focus on accessibility (A11y)
  - flexible grid and UI components
  - modular – include only what you need
- 🔴 **disadvantages:**
  - smaller community than Bootstrap
  - slightly steeper learning curve
  - can be heavy if not customized

- **Kitchen Sink**

Example

Bulma

- bulma.io, MIT License
- modern CSS-only framework based on Flexbox, providing responsive grid, UI components, and utilities
- 🟢 **advantages:**
  - lightweight and easy to use, no JavaScript required
  - modern Flexbox-based grid system
  - clean, minimalistic design
  - modular – import only what you need
- 🔴 **disadvantages:**
  - lacks built-in JS components (requires custom JS for interactivity)
  - smaller community compared to Bootstrap
  - may require writing JS for interactive elements

- **components**, **comparison with Bootstrap**

Example

Materialize

- materializecss.com, MIT License
- front-end framework based on Google's Material Design, providing CSS, JavaScript components, responsive grid, and UI elements
- 🟢 **advantages:**
  - built-in Material Design aesthetics
  - responsive grid system and prebuilt UI components
  - includes JavaScript components (modals, dropdowns, tooltips, sidenav, etc.)
  - easy to use for prototyping and consistent design
- 🔴 **disadvantages:**
  - can be heavier due to combined CSS + JS
  - design is opinionated – may require overrides for custom look
  - smaller community than Bootstrap

- **components**

Example

Type 2: Utility-based Frameworks

- provides small, reusable classes that define a single property (e.g., `text-center`, `p-4`, `bg-gray-200`)
- **Tailwind CSS**
- 🟢 **advantages:**
  - high flexibility – build custom UI without writing new CSS
  - minimizes the need to write custom styles
- 🔴 **disadvantages:**
  - can lead to long class lists in HTML
  - requires careful planning of naming and structure

Tailwind CSS

- tailwindcss.com, MIT License
- utility-first CSS framework providing small, composable classes for styling directly in HTML; highly configurable and responsive
- 🟢 **advantages:**
  - high flexibility – build custom designs without writing custom CSS
  - utility classes promote consistency and reuse
  - responsive design and pseudo-class variants built-in (sm:, md:, hover:, etc.)
  - large community and rich ecosystem (plugins, components, templates)
- 🔴 **disadvantages:**
  - long class lists in HTML can reduce readability
  - steeper learning curve for beginners unfamiliar with utility-first approach
  - requires build tools (PostCSS, CLI, or CDN) for best performance

- **styling with utility classes**

Example

Drawbacks

- **bloat / Large size** – many unused styles increase CSS size
- **learning curve** – must learn framework classes and grid system
- **reduced flexibility** – prebuilt components can limit custom design
- **uniform look** – default components make sites look similar
- **dependency** – tied to framework versions and updates
- **HTML clutter** – long class lists reduce readability
- **performance** – extra unused styles may slow down pages

CSS Preprocessors

Variables, Scopes, Mixins

Motivation

- scripting / meta languages that compile to CSS
- extends CSS with variables, nesting, mixins, functions, and more

- 🟢 **goals:**
  - variables for consistent values (colors, fonts, spacing)
  - nesting for clearer structure
  - mixins and functions for reusable code
  - logic and operations (loops, conditionals)
  - modular code via imports

- **Sass** / SCSS – most widely used, mature, supports variables, nesting, mixins, functions
- **LESS** – similar to Sass, integrates well with JavaScript
- **Stylus** – highly flexible, minimal syntax, supports both indented and CSS-like style

Variables

Sass variables

```sass
$primary-color: #3498db;
$text-color: #333;
$spacing: 16px;

body {
  color: $text-color;
  font-family: Arial, sans-serif;
}

.button {
  background-color: $primary-color;
  padding: $spacing;
  border-radius: 4px;
}
```

```css
body {
  color: #333;
  font-family: Arial, sans-serif;
}

.button {
  background-color: #3498db;
  padding: 16px;
  border-radius: 4px;
}
```

- Sass variables are compile-time only, while CSS variables work at runtime and can change dynamically in the browser

Nesting

- **Sass nesting**

```sass
.nav {
  background: #333;

  a {
    color: white;

    &:hover {
      color: yellow;
    }
  }
}
```

```css
.nav { background: #333; }
.nav a { color: white; }
.nav a:hover { color: yellow; }
```

- Sass nesting is compiled and more permissive, while CSS nesting is native but has stricter syntax rules.
  - **CSS Nesting Level 1** (Editor's Draft) – MDN

Mixins

- **Sass mixins**

```sass
@mixin flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}

.container {
  @include flex-center;
  height: 200px;
  background-color: #3498db;
}
```

```css
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 200px;
  background-color: #3498db;
}
```

- mixins exist in preprocessors like Sass, but CSS has no native mixins

Advanced CSS Features

Filters, Transformations, Transitions, Animations

Motivation

- modern CSS evolves through **independent modules and levels** rather than big version numbers like CSS3 or CSS4.
- **W3C current work**

- **vendor prefixes**
  - used to enable experimental or draft features before standardization
  - `-moz-` Firefox
  - `-ms-` IE
  - `-webkit-` Chrome, Safari
  - `-o-` Opera (obsolete)

New Features: Container Queries

```html
<div class="post">
  <div class="card">
    <h2>Card title</h2>
    <p>Card content</p>
  </div>
</div>
```

- create a **containment context**, use Container Query Length Units

```css
.post {
  container-type: inline-size; /* containment context */
}

@container (width > 700px) {
  .card h2 {
    font-size: max(1.5em, 1.23em + 2cqi);
    /* cqi - 1% of a query container's inline size */
  }
}
```

New Features: Subgrid

```html
<div class="grid">
  <div class="item">
    <div class="subitem"></div>
  </div>
</div>
```

```css
.grid { /* red */
  display: grid;
  grid-template-columns: repeat(9, 1fr);
}

.item { /* blue */
  display: grid;
  grid-column: 2 / 7;
  grid-template-columns: subgrid;
}

.subitem { /* green */
  grid-column: 3 / 6;
}
```

1

2

3

4

New Features…

- CSS Nesting
- `:has()` pseudo-class
- Style Queries
- `:scope` / `@scope`
- Cascade Layers (`@layer`)
- `@property` rule
- `color()` – support for HWB/LAB/LCH
- Color mixing and blending – e.g., `color-mix()`
- New viewport units (`svh`, `lvh`, `dvh`)
- CSS Math & trigonometry – e.g., `sin()`, `cos()`
- …

filter

- enables simple visual effects directly in CSS without graphics editing tools.

```
img {
  filter:
    grayscale(40%)
    sepia(.8)
    contrast(120%)
    brightness(110%)
    drop-shadow(0px 5px 10px black);
}

img:hover {
  filter: none;
}
```

Example

transform

- lets you rotate, scale, translate, and skew elements in 2D and 3D space.

```
img {
  transform:
    rotate(15deg)
    scale(.5)
    translateX(50%)
    skewY(5deg);
}

img:hover {
  transform: none;
}
```

Example

transition

- lets you smoothly animate changes to CSS properties over a specified duration.
- `transition: property duration timing-function delay;`

```
img {
  filter: … ;
  transform: … ;
  transition: all .5s ease;
}

img:hover {
  filter: none;
  transform: none;
}
```

Example

animation, @keyframes

- let you create repeated or timed changes to element properties **without JavaScript**.
- `animation: name duration timing-function delay iteration-count direction;`

```
@keyframes bounce {
  0%, 100% {
    transform: translateY(0);
  }
  20% {
    transform: translateX(200px);
    width: 200px;
  }
  50% {
    transform: translate(300px, 100px)
               rotate(360deg);
    width: 400px;
  }
}
img {
  width: 200px;
  animation: bounce 5s infinite;
}
```

Example

Summary

links

Key Information

- media types: `screen`, `print`, `all`
- media-features: `min/max-width/height`
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">`
- `break-before`, `break-after`, `break-inside`, `@page`

- component-based vs. utility-based frameworks

- CSS preprocessors: variables, nesting, mixins

- CSS current work: https://www.w3.org/Style/CSS/current-work

To be continued…

JavaScript – Basic Concepts of Dynamic Pages